



Université Sidi Mohamed Ben Abdellah – Fès

École Nationale des Sciences Appliquées de Fès (ENSAF)

Année Académique : **2025–2026**

Filière : **Génie du Développement Numérique et Cybersécurité
(GDNC)**

Module : **Développement d'Applications sécurisées**

Déploiement d'un honeypot SSH et confinement AppArmor/Seccomp avec supervision ELK

Compte rendu de projet

Étudiant	Saad Nachit
Professeur	Pr. Mohamed AIRAJ
Date	05/12/2025

Table des matières

1	Introduction	3
2	Environnement et architecture	4
2.1	Environnement technique	4
2.2	Architecture globale	4
3	Honeypot SSH en Python	5
3.1	Implémentation avec Paramiko	5
3.2	Fichier de logs JSONL	5
3.3	Normalisation en fichier JSONL compact	6
4	Confinement AppArmor	7
4.1	Objectifs	7
4.2	Création du profil AppArmor	7
5	Filtrage Seccomp	9
5.1	Objectifs	9
5.2	Implémentation du filtrage Seccomp	9
5.2.1	Politique Seccomp utilisée	9
5.2.2	Appels système bloqués	10
5.2.3	Activation au démarrage	11
6	Mise en place de la stack Elastic (ELK)	12
6.1	Installation et configuration d'Elasticsearch	12
6.2	Pipeline Logstash pour les logs du honeypot	12
6.3	Vérification des indices Elasticsearch	14
7	Configuration de Kibana et visualisation	16
7.1	Configuration de base	16
7.2	Data view honeypot-ssh-clean	16
7.3	Visualisations et dashboard	16
8	Difficultés rencontrées et solutions	18
8.1	Permissions sur les fichiers de logs	18
8.2	Gestion du syncedb	18
8.3	Erreurs de parsing JSON (_jsonparsefailure)	18
8.4	Configuration de Kibana	19
8.5	Intégration Seccomp	19
9	Bonus : Détection de brute-force et alertes	20

9.1	Objectif	20
9.2	Filtre Logstash de détection de brute-force	20
9.3	Alerte via webhook (output HTTP)	21
9.4	Tests de brute-force et validation	22
9.4.1	Simulation d'attaque avec <code>sshpas</code>	22
9.4.2	Observation dans Logstash	22
9.4.3	Vérification dans Elasticsearch et Kibana	24
9.5	Tests avec Hydra et vérification du confinement	25
9.5.1	Tentatives avec Hydra	25
9.5.2	Vérification du profil AppArmor en situation d'attaque	25
10	Conclusion	27

1. Introduction

L'objectif de ce projet est de développer un honeypot SSH en Python, de le confiner à l'aide de mécanismes de sécurité du noyau (AppArmor et Seccomp) et de centraliser toutes les tentatives d'intrusion dans une stack Elastic (Elasticsearch, Logstash, Kibana), afin d'obtenir une visualisation en quasi temps réel des activités malveillantes.

Plus précisément, le cahier des charges demande :

- d'implémenter un honeypot SSH avec la bibliothèque **paramiko**, simulant un serveur SSH et enregistrant les tentatives de connexion (logins, mots de passe, commandes) ;
- de définir un profil AppArmor restreignant l'accès réseau et fichier du honeypot ;
- d'appliquer un filtre Seccomp limitant les appels système autorisés ;
- de configurer Logstash pour parser les logs JSON du honeypot et les indexer dans Elasticsearch ;
- de créer un dashboard Kibana (par exemple fréquence des tentatives, IPs sources, etc.) ;
- en bonus, de mettre en place une détection simple de brute-force.

Dans ce rapport, le confinement est assuré par **AppArmor** (aucune politique SELinux n'est utilisée), complété par un filtrage **Seccomp**. La stack ELK permet ensuite l'analyse et la visualisation structurée des événements.

2. Environnement et architecture

2.1 Environnement technique

Le projet a été réalisé sur une machine virtuelle **Kali Linux** (amd64) sous VirtualBox. Les principaux composants utilisés sont :

- **Honeypot SSH** : script Python basé sur **paramiko**, installé dans `/opt/ssh_honeypot`.
- **AppArmor** : profil spécifique pour le honeypot.
- **Seccomp** : filtrage des appels système via une bibliothèque Python.
- **Elasticsearch 8.19.7** : moteur d'indexation des logs.
- **Logstash 8.19.7** : pipeline d'ingestion et de transformation des logs.
- **Kibana 8.19.7** : interface web d'exploration et de visualisation.

Tous les composants de la stack Elastic sont exécutés sur la même machine. Elasticsearch et Kibana sont configurés pour n'écouter que sur `127.0.0.1` afin de ne pas exposer les interfaces d'administration sur le réseau.

2.2 Architecture globale

L'architecture logique du système est la suivante :

1. Un client (ou un attaquant simulé) se connecte au honeypot SSH.
2. Le honeypot enregistre chaque événement (authentification, commande, erreur) dans un fichier texte JSONL.
3. Logstash lit ce fichier, parse les lignes JSON, enrichit les événements et les envoie à Elasticsearch.
4. Elasticsearch indexe les documents dans des indices quotidiens dédiés au honeypot.
5. Kibana se connecte à Elasticsearch et permet l'exploration ainsi que la création de dashboards.

En parallèle, AppArmor et Seccomp réduisent au maximum les privilèges du honeypot, afin que sa compromission ait un impact limité.

3. Honeypot SSH en Python

3.1 Implémentation avec Paramiko

Le honeypot SSH est basé sur la bibliothèque `paramiko`, qui permet de créer un serveur SSH en Python. Le script principal (`/opt/ssh_honeypot/ssh_honeypot.py`) effectue les opérations suivantes :

- génération et chargement d'une clé d'hôte (host key) ;
- écoute sur un port TCP dédié (par exemple 2222) ;
- gestion des tentatives d'authentification (utilisateur / mot de passe) ;
- ouverture de sessions interactives ou exécution de commandes ;
- journalisation d'événements structurés (JSON).

Les principaux types d'événements enregistrés sont :

- **auth_attempt** : tentative de connexion avec les champs suivants :
 - **src_ip**, **src_port** : IP/port source ;
 - **username**, **password** ;
 - **success** : booléen indiquant si l'authentification a réussi ;
 - **timestamp** : horodatage ISO 8601.
- **session_start** et **session_close** : ouverture et fermeture de session.
- **shell_command** : commande saisie dans un shell interactif (par exemple `ls`, `whoami`, `exit`).
- **exec_command** : commande exécutée via un canal `exec`.
- **server_error** : erreur interne au serveur (exception Python, reset de connexion, etc.).
- **seccomp** : statut de l'initialisation du filtre Seccomp.
- **hostkey** : création de la clé d'hôte.

3.2 Fichier de logs JSONL

Les événements sont écrits dans le fichier :

```
/var/log/honeypot/ssh.jsonl
```

Chaque ligne correspond à un objet JSON complet. Exemple :

```
1 {"event": "auth_attempt", "src_ip": "127.0.0.1", "src_port": 45084,
2   "username": "test", "password": "kali", "success": true,
3   "timestamp": "2025-11-14T17:21:09Z"}
4
5 {"event": "shell_command", "src_ip": "127.0.0.1", "src_port": 50606,
6   "username": "test", "command": "ls",
7   "timestamp": "2025-11-14T17:27:28Z"}
8
9 {"event": "server_error",
```

```

10  "error":"[Errno 104] Connection reset by peer",
11  "trace":"Traceback (most recent call last): ...",
12  "timestamp":"2025-11-14T17:45:35Z"}

```

Listing 3.1 Exemples d'événements bruts du honeypot

Pour s'assurer de la validité JSON du fichier et éviter des problèmes de parsing, le contenu est vérifié avec l'outil `jq` :

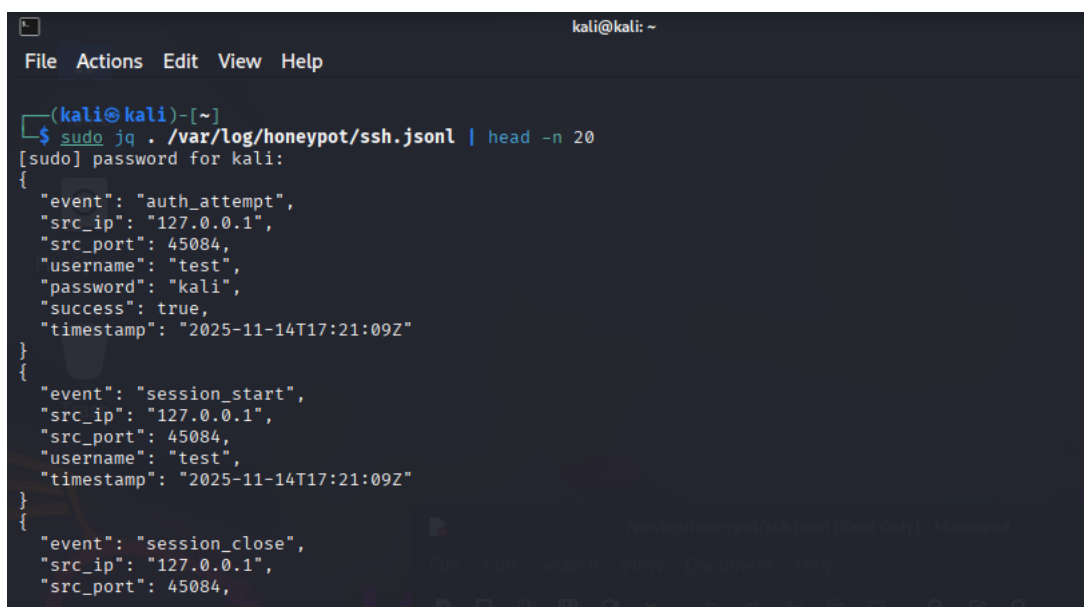
```

1  sudo jq . /var/log/honeypot/ssh.jsonl | head -n 20

```

Listing 3.2 Validation JSON des logs

La sortie de cette commande a été capturée (voir capture d'écran ci-après) afin d'illustrer la bonne structuration des objets JSON présents dans le fichier.



```

kali@kali: ~
File Actions Edit View Help

(kali@kali)-[~]
$ sudo jq . /var/log/honeypot/ssh.jsonl | head -n 20
[sudo] password for kali:
{
  "event": "auth_attempt",
  "src_ip": "127.0.0.1",
  "src_port": 45084,
  "username": "test",
  "password": "kali",
  "success": true,
  "timestamp": "2025-11-14T17:21:09Z"
}
{
  "event": "session_start",
  "src_ip": "127.0.0.1",
  "src_port": 45084,
  "username": "test",
  "timestamp": "2025-11-14T17:21:09Z"
}
{
  "event": "session_close",
  "src_ip": "127.0.0.1",
  "src_port": 45084,

```

Figure 3.1 Affichage des premières lignes du fichier `ssh.jsonl` après validation avec `jq`.

3.3 Normalisation en fichier JSONL compact

Logstash est plus simple à configurer lorsqu'on dispose d'un fichier avec une structure JSON par ligne, sans espaces ni sauts inutiles. Un fichier normalisé `ssh_single_line.jsonl` est donc généré :

```

1  sudo jq -c . /var/log/honeypot/ssh.jsonl \
2  | sudo tee /var/log/honeypot/ssh_single_line.jsonl > /dev/null

```

Listing 3.3 Génération du fichier normalisé

Ce fichier est utilisé comme source principale par Logstash.

4. Confinement AppArmor

4.1 Objectifs

AppArmor apporte un confinement basé sur des chemins de fichiers et des règles de type « profil ». L'objectif est d'autoriser uniquement ce qui est strictement nécessaire au honeypot et de refuser le reste, afin de limiter l'impact d'une compromission éventuelle.

Plus précisément, il s'agit de :

- autoriser uniquement l'accès aux fichiers du honeypot et aux logs ;
- interdire explicitement `/etc/**` en lecture ;
- limiter l'accès au réseau aux ports nécessaires ;
- enregistrer toutes les violations éventuelles dans les logs du noyau.

4.2 Création du profil AppArmor

Un profil AppArmor est créé, `/etc/apparmor.d/honeypot-ssh`. Voilà le profil :

```
1 #include <tunables/global>
2 profile honeypot-ssh flags=(complain) {
3     include <abstractions/base>
4     include <abstractions/python>
5
6     # Réseau TCP (v4/v6) pour SSH
7     network inet tcp,
8     network inet6 tcp,
9
10    # Lire le code du projet
11    /opt/ssh_honeypot/** r,
12    /opt/ssh_honeypot/venv/** mr,
13
14    # créer les logs
15    /var/log/honeypot/ rw,
16    /var/log/honeypot/* rw,
17    /opt/ssh_honeypot/keys/ rw,
18    /opt/ssh_honeypot/keys/* rw,
19
20    # Bibliothèques dynamiques
21    /usr/lib/** mr,
22    /lib/** mr,
23    /usr/lib/x86_64-linux-gnu/** mr,
24
```



```
25 # Rduction de surface
26 deny /etc/** r,
27 deny /proc/** r,
28 deny /home/** r,
29 }
```

Listing 4.1 Profil AppArmor pour le honeypot

Le profil est chargé puis activé :

```
1 sudo apparmor_parser -r /etc/apparmor.d/honeypot-ssh
2 sudo aa-enforce honeypot-ssh
```

Les violations de profil sont visibles dans `/var/log/syslog` ou `/var/log/kern.log` et permettent d'ajuster finement la configuration si nécessaire. Des essais d'accès à `/etc/passwd` ou à d'autres chemins sensibles depuis le code du honeypot permettent de vérifier le bon fonctionnement du confinement.

5. Filtrage Seccomp

5.1 Objectifs

Seccomp permet de limiter les appels système que le processus peut exécuter. L'idée est que le honeypot n'a besoin que d'un nombre restreint de syscalls (création de sockets, lecture/écriture, gestion des signaux et du temps, etc.), et que tout le reste doit être interdit.

Ainsi, même si un attaquant parvient à injecter du code dans le processus, il ne pourra pas par exemple exécuter un nouveau binaire avec `execve` ou manipuler directement des fichiers système.

5.2 Implémentation du filtrage Seccomp

Le projet intègre un mécanisme de réduction de surface d'attaque basé sur `seccomp-bpf`. L'implémentation repose sur la bibliothèque `pyseccomp`, chargée dynamiquement au démarrage du honeypot :

```
1 try:
2     import pyseccomp as seccomp
3 except Exception:
4     seccomp = None
```

Listing 5.1 Chargement optionnel de pyseccomp

Si la bibliothèque n'est pas disponible (par exemple sur une distribution ne fournissant pas `libseccomp` ou le module Python), un événement est enregistré dans les logs et le honeypot continue sans filtrage :

```
1 {"event": "seccomp", "status": "not_available"}
```

Listing 5.2 Log généré en absence de pyseccomp

5.2.1 Politique Seccomp utilisée

Lorsque `pyseccomp` est présent, une politique « default-allow » est appliquée, dans laquelle seuls certains appels système critiques sont explicitement bloqués. La fonction responsable de l'application du filtre est :

```
1 def apply_seccomp():
2     if not seccomp:
3         log_event({"event": "seccomp", "status": "not_available"})
4         return False
5     try:
6         # Politique par défaut : autoriser tout
7         f = seccomp.SyscallFilter(seccomp.SCMP_ACT_ALLOW)
8
```

```

9      # Action de blocage : renvoyer EPERM (errno 13)
10     deny = seccomp.SCMP_ACT_ERRNO(13)
11
12     # Liste noire (blacklist) des appels systme interdits
13     blacklist = [
14         "execve", "execveat",
15         "fork", "vfork",
16         "ptrace",
17         "kexec_load", "kexec_file_load",
18         "init_module", "finit_module", "delete_module",
19         "mount", "umount2", "pivot_root",
20         "reboot",
21     ]
22
23     # Ajout des rgles de blocage
24     for name in blacklist:
25         try:
26             f.add_rule(deny, name)
27         except Exception:
28             pass
29
30     # Activation du filtre dans le processus
31     f.load()
32
33     log_event({
34         "event": "seccomp",
35         "status": "enabled",
36         "policy": "default-allow+deny_critical_exec/sysadmin"
37     })
38
39     return True
40 except Exception as e:
41     log_event({"event": "seccomp", "status": "enable_failed", "error": str(e)})
42     return False

```

Listing 5.3 Application du filtre Seccomp issu du projet

5.2.2 Appels système bloqués

La politique empêche spécifiquement :

- toute création de processus : `execve`, `fork`, `vfork`,
- tout chargement de module noyau : `init_module`, `finit_module`,
- toute manipulation du système de fichiers root : `mount`, `pivot_root`,
- toute tentative de redémarrage : `reboot`,
- toute capacité de débogage : `ptrace`.

Cette approche permet au honeypot d'exécuter uniquement les opérations nécessaires (Networking, Paramiko, E/S simples), tout en empêchant les actions dangereuses susceptibles d'être tentées

par un attaquant via une vulnérabilité éventuelle.

5.2.3 Activation au démarrage

Le filtrage Seccomp est appliqué au moment du lancement du serveur SSH :

```
1  apply_seccomp()
```

En cas d'échec, un log dédié permet de détecter rapidement un problème de configuration.

6. Mise en place de la stack Elastic (ELK)

6.1 Installation et configuration d'Elasticsearch

Elasticsearch est installé via les paquets Debian. Le fichier `/etc/elasticsearch/elasticsearch.yml` est adapté comme suit (extraits) :

```
— network.host: "127.0.0.1"
— http.port: 9200
— discovery.type: single-node
— xpack.security.enabled: false
```

Le service est démarré et vérifié :

```
1 sudo systemctl enable elasticsearch
2 sudo systemctl start elasticsearch
3 curl -s http://127.0.0.1:9200 | jq .
```

6.2 Pipeline Logstash pour les logs du honeypot

Le pipeline spécifique est défini dans `/etc/logstash/conf.d/honeypot-ssh.conf`. Il suit la structure `input / filter / output` :

```
1 input {
2   file {
3     path => "/var/log/honeypot/ssh_single_line.jsonl"
4     start_position => "beginning"
5     sincedb_path => "/dev/null"
6     codec => "plain"
7   }
8 }
9
10 filter {
11   # Dcoder la ligne JSON contenue dans [message] vers [doc]
12   json {
13     source => "message"
14     target => "doc"
15   }
16
17   # Timestamp dingestion
18   mutate {
19     add_field => { "ingest_time" => "%{@timestamp}" }
20   }
21 }
```

```

21
22 # Recopier quelques champs utiles de [doc] vers la racine
23 if [doc][event] {
24     mutate { add_field => { "event" => "%{[doc][event]}" } }
25 }
26 if [doc][src_ip] {
27     mutate {
28         add_field => {
29             "src_ip" => "%{[doc][src_ip]}"
30             "source.ip" => "%{[doc][src_ip]}"
31         }
32     }
33 }
34 if [doc][username] {
35     mutate {
36         add_field => {
37             "username" => "%{[doc][username]}"
38             "user.name" => "%{[doc][username]}"
39         }
40     }
41 }
42 if [doc][src_port] {
43     mutate { convert => { "[doc][src_port]" => "integer" } }
44 }
45
46 # Gérer le timestamp du honeypot
47 if [doc][timestamp] {
48     mutate { rename => { "[doc][timestamp]" => "[doc][honeypot_timestamp]" } }
49     date {
50         match => ["[doc][honeypot_timestamp]", "ISO8601"]
51         target => "@timestamp"
52         timezone => "UTC"
53     }
54 }
55
56 # Taguer les erreurs serveur
57 if [doc][event] == "server_error" {
58     mutate { add_tag => ["honeypot_error"] }
59 }
60
61 # Détection brute-force : 5 tentatives/minute par IP
62 if [doc][event] == "auth_attempt" {
63     aggregate {
64         task_id => "%{[doc][src_ip]}"
65         code => "
66             map['count'] ||= 0
67             map['first_ts'] ||= event.get('@timestamp')
68             map['count'] += 1

```

```

69
70     if map['count'] >= 5
71         event.set('bruteforce', true)
72         event.set('bruteforce_count', map['count'])
73         event.set('bruteforce_ip', event.get('[doc][src_ip]'))
74         event.set('bruteforce_first_ts', map['first_ts'])
75     end
76     "
77     timeout_task_id_field => "[doc][src_ip]"
78     timeout => 60
79 }
80 }
81 }
82
83 output {
84     elasticsearch {
85         hosts => ["http://127.0.0.1:9200"]
86         index => "honeypot-ssh-clean-%{+YYYY.MM.dd}"
87     }
88
89     stdout {
90         codec => rubydebug
91     }
92 }

```

Listing 6.1 Pipeline Logstash pour le honeypot

Quelques points importants :

- `sincedb_path => "/dev/null"` permet de forcer la relecture complète du fichier à chaque redémarrage (pratique pour les tests).
- Le filtre `json` est utilisé au niveau du filtre pour avoir un contrôle précis sur la structure des champs.
- Le champ `@timestamp` est recalculé à partir du `timestamp` du honeypot.

6.3 Vérification des indices Elasticsearch

Après démarrage de Logstash :

```

1 sudo systemctl restart logstash
2 sudo journalctl -u logstash -f

```

On vérifie la création des indices :

```

1 curl -s "http://127.0.0.1:9200/_cat/indices?v" | grep honeypot-ssh
2
3 curl -s "http://127.0.0.1:9200/honeypot-ssh-clean-*/_search?size=3" \
4 | jq '.hits.hits[0]._source'

```

Listing 6.2 Indices Elasticsearch du honeypot

On obtient :

```
kali@kali: ~  
File Actions Edit View Help  
kali@kali: ~  
$ curl -s "http://127.0.0.1:9200/_cat/indices?v" | grep honeypot-ssh  
  
curl -s "http://127.0.0.1:9200/honeypot-ssh-clean-*/_search?size=3" \  
| jq '.hits.hits[0]._source'  
yellow open honeypot-ssh-2025.11.16 46.1kb 46.1kb owgUFiwwSdygzUcq2vPGUQ 1 1  
yellow open honeypot-ssh-clean-2025.11.15 178.8kb 178.8kb Yq9GoAwGS02CE00Dm6hf0Q 1 1  
yellow open honeypot-ssh-clean-2025.11.14 272.5kb 272.5kb ScLUGWRuRmCGJjdkeZ7j-Q 1 1  
yellow open honeypot-ssh-clean-2025.11.16 113kb 113kb c4LhCEa2QnqjMH4uHX5ZmA 1 1  
yellow open honeypot-ssh-clean-2025.11.18 9.4kb 9.4kb GZ6ddqylS72AM8Xw_bgc-g 1 1  
{  
  "message": "{\n\"event\": \"auth_attempt\", \"src_ip\": \"127.0.0.1\", \"src_port\": 45084, \"username\": \"test\", \"password\": \"kali\", \"success\": true, \"timestamp\": \"2025-11-14T17:21:09Z\"}",  
  "@timestamp": "2025-11-14T17:21:09.000Z",  
  "username": "test",  
  "source.ip": "127.0.0.1",  
  "@version": "1",  
  "user.name": "test",  
  "doc": {  
    "password": "kali",  
    "username": "test",  
    "event": "auth_attempt",  
    "src_port": 45084,  
    "success": true,  
    "src_ip": "127.0.0.1",  
    "honeypot_timestamp": "2025-11-14T17:21:09Z"  
  },  
  "host": {  
    "name": "kali"  
  },  
  "log": {  
    "file": {  
      "path": "/var/log/honeypot/ssh_single_line.jsonl"  
    }  
  },  
  "ingest_time": "2025-11-16T12:33:25.823233386Z",  
  "event": [  
    "original",  
    "{\n\"event\": \"auth_attempt\", \"src_ip\": \"127.0.0.1\", \"src_port\": 45084, \"username\": \"test\", \"password\": \"kali\", \"success\": true, \"timestamp\": \"2025-11-14T17:21:09Z\"}",  
  ],  
  "auth_attempt"  
],  
  "src_ip": "127.0.0.1"  
}
```

Figure 6.1 Résultat de la commande de vérification des indices Elasticsearch pour le honeypot.

Les documents contiennent des champs structurés (`doc.event`, `doc.src_ip`, `doc.username`, `doc.command`, etc.), ainsi que `source.ip` et `user.name` compatibles ECS.

7. Configuration de Kibana et visualisation

7.1 Configuration de base

Kibana est installé et configuré via `/etc/kibana/kibana.yml` :

```
1 server.host: "127.0.0.1"
2 server.port: 5601
3 elasticsearch.hosts: ["http://127.0.0.1:9200"]
```

Listing 7.1 Extraits de configuration Kibana

Le service est démarré :

```
1 sudo systemctl enable kibana
2 sudo systemctl start kibana
3 curl -I http://127.0.0.1:5601
```

Une fois l'interface accessible, Kibana est ouvert dans le navigateur à l'adresse <http://127.0.0.1:5601>.

7.2 Data view honeypot-ssh-clean

Dans la section *Discover*, un data view spécifique est créé :

- **Name** : honeypot-ssh-clean
- **Index pattern** : honeypot-ssh-clean-*
- **Time field** : @timestamp

En sélectionnant ce data view et une période suffisamment large (par exemple *Last 1 year*), on visualise :

- l'histogramme temporel des événements ;
- la liste des documents, avec les champs `doc.event`, `doc.src_ip`, `doc.username`, `doc.command`, etc.

7.3 Visualisations et dashboard

Plusieurs visualisations sont créées à l'aide de Kibana Lens :

- **Timeline des événements** : histogramme du Count en fonction de @timestamp.
- **Événements par type** : diagramme en barres Count vs `doc.event.keyword`.
- **Top IP sources** : diagramme en barres Count vs `source.ip`.

- **Top usernames** : diagramme `Count` vs `user.name`.
- **Top commandes** : `Count` vs `doc.command.keyword`, filtré sur `doc.event:("shell_command"`
OR `"exec_command")`.

Ces visualisations sont ajoutées à un tableau de bord intitulé par exemple `Dashboard Honeypot SSH`. Ce dashboard fournit une vue d'ensemble de l'activité observée sur le honeypot.

8. Difficultés rencontrées et solutions

8.1 Permissions sur les fichiers de logs

Au départ, Logstash ne pouvait pas lire `/var/log/honeypot/ssh_single_line.jsonl` en raison de permissions insuffisantes. Des erreurs `Permission denied` apparaissaient.

Solution : ajuster les droits du répertoire :

```
1 sudo chmod o+rx /var/log/honeypot
```

tout en gardant des droits raisonnablement restrictifs sur les fichiers.

8.2 Gestion du `sincedb`

Le plugin `file` de Logstash utilise un fichier `sincedb` pour mémoriser la position de lecture. Au cours des tests, cela empêchait la relecture complète des fichiers après modification.

Solution : pour le TP, utiliser :

```
1 sincedb_path => "/dev/null"
```

ce qui force la relecture du fichier à chaque démarrage.

8.3 Erreurs de parsing JSON (`_jsonparsefailure`)

Des tags `_jsonparsefailure` apparaissaient, et les documents étaient stockés avec un champ `message` contenant le JSON brut, ce qui rendait l'exploitation difficile.

Causes :

- présence temporaire de lignes non JSON dans le fichier de logs (erreur de manipulation) ;
- utilisation initiale d'un codec JSON inadapté ;
- option `skip_invalid_json` non supportée par la version du codec.

Solutions :

- régénérer le fichier à partir de `ssh.jsonl` avec `jq -c` pour garantir une ligne JSON valide par événement ;
- basculer sur un input texte simple et utiliser le filtre `json` dans la section `filter` ;
- supprimer l'option `skip_invalid_json`.

8.4 Configuration de Kibana

Kibana affiche des avertissements concernant les clés d'encryption (`xpack.encryptedSavedObjects.encryptionKey`, `xpack.reporting.encryptionKey`, `xpack.security.encryptionKey`). Dans un contexte de laboratoire, Kibana peut générer des clés aléatoires au démarrage ; les fonctionnalités principales (Discover, Visualize, Dashboard) restent disponibles.

8.5 Intégration Seccomp

L'intégration de Seccomp a entraîné des problèmes de compatibilité avec certaines versions de bindings Python, générant des messages d'erreur du type :

```
1 "error": "module 'pyseccomp' has no attribute 'SCMP_ACT_ALLOW'"
```

Solution : journaliser explicitement les erreurs Seccomp dans les logs du honeypot (`event:"seccomp"`) pour les détecter rapidement, puis adapter la bibliothèque ou la version utilisée.

9. Bonus : Détection de brute-force et alertes

9.1 Objectif

En complément de la collecte et de la visualisation des événements du honeypot, un mécanisme de détection de brute-force a été implémenté. L'objectif est :

- de détecter automatiquement les adresses IP qui effectuent un nombre anormalement élevé de tentatives SSH (seuil : 5 tentatives par minute et par IP) ;
- de marquer ces événements côté Logstash/Elasticsearch pour faciliter les recherches et les visualisations dans Kibana ;
- de prévoir l'envoi d'alertes via un webhook HTTP lorsque ce seuil est dépassé ;
- de tester ce dispositif à l'aide d'outils d'attaque en ligne de commande (Hydra, **sshpass**) tout en restant dans le cadre du confinement AppArmor.

9.2 Filtre Logstash de détection de brute-force

La détection de brute-force est réalisée dans Logstash à l'aide du filtre **aggregate**. Celui-ci maintient un compteur par adresse IP source et marque les événements dès que le nombre de tentatives d'authentification atteint ou dépasse 5 sur une fenêtre glissante de 60 secondes.

Le bloc suivant est ajouté dans la section **filter** du pipeline `/etc/logstash/conf.d/honeypot-ssh.conf` :

```
1 # Dtection brute-force : 5 tentatives/minute par IP
2 if [doc][event] == "auth_attempt" {
3   aggregate {
4     task_id => "%{[doc][src_ip]}"
5     code => "
6       map['count'] ||= 0
7       map['first_ts'] ||= event.get('@timestamp')
8       map['count'] += 1
9
10      if map['count'] >= 5
11        event.set('bruteforce', true)
12        event.set('bruteforce_count', map['count'])
13        event.set('bruteforce_ip', event.get('[doc][src_ip]'))
14        event.set('bruteforce_first_ts', map['first_ts'])
15      end
16    "
17    timeout_task_id_field => "[doc][src_ip]"
18    timeout => 60
19  }
```

```
20 }
```

Listing 9.1 Filtre Logstash de détection de brute-force

Fonctionnement :

- `task_id` est l'adresse IP source (`doc.src_ip`);
- `map['count']` est le compteur de tentatives pour cette IP;
- `map['first_ts']` mémorise le premier `@timestamp` observé dans la fenêtre;
- si `map['count'] >= 5`, l'événement courant reçoit les champs :
 - `bruteforce:true`;
 - `bruteforce_count` : nombre total de tentatives dans la fenêtre;
 - `bruteforce_ip` : adresse IP incriminée;
 - `bruteforce_first_ts` : horodatage de la première tentative de la vague.
- après 60 secondes sans nouvelle tentative pour cette IP, l'état associé est purgé (via `timeout`).

9.3 Alerte via webhook (output HTTP)

Pour aller plus loin, il est possible d'envoyer une alerte vers un système externe (par exemple un petit serveur Flask, un SIEM, un outil de messagerie, etc.) via une simple requête HTTP POST. Le bloc suivant illustre un output `http` conditionnel sur le champ `bruteforce` :

```
1  output {
2    elasticsearch {
3      hosts => ["http://127.0.0.1:9200"]
4      index => "honeypot-ssh-clean-%{+YYYY.MM.dd}"
5    }
6
7    stdout {
8      codec => rubydebug
9    }
10
11   # --- Alerte brute-force via webhook (exemple) ---
12   if [bruteforce] == true {
13     http {
14       url => "http://127.0.0.1:8080/webhook" # URL du webhook adapter
15       http_method => "post"
16       format => "json"
17       content_type => "application/json"
18       mapping => {
19         "type" => "bruteforce_alert"
20         "ip" => "%{bruteforce_ip}"
21         "count" => "%{bruteforce_count}"
22         "first_ts" => "%{bruteforce_first_ts}"
23         "last_ts" => "%{@timestamp}"
24         "username" => "%{[doc][username]}"
25       }
26     }
27   }
```

```

26     }
27 }
28 }

```

Listing 9.2 Exemple d'output HTTP pour les alertes brute-force

9.4 Tests de brute-force et validation

9.4.1 Simulation d'attaque avec sshpass

Afin de provoquer des vagues de tentatives d'authentification répétées, un script bash simple a été utilisé avec `sshpass`. L'objectif est d'exécuter une dizaine de connexions SSH successives vers le honeypot depuis la même adresse IP (ici 127.0.0.1) :

```

1 for i in $(seq 1 10); do
2     sshpass -p 'kali' \
3     ssh -o PreferredAuthentications=password \
4         -o PubkeyAuthentication=no \
5         -o StrictHostKeyChecking=no \
6         -p 2222 test@127.0.0.1 'echo test' || true
7 done

```

Listing 9.3 Boucle de brute-force simple avec sshpass

Chaque itération génère au moins un événement `auth_attempt` dans `/var/log/honeypot/ssh.jsonl`, avec un `timestamp` très rapproché (moins d'une minute). Le fichier normalisé est ensuite régénéré :

```

1 sudo jq -c . /var/log/honeypot/ssh.jsonl \
2 | sudo tee /var/log/honeypot/ssh_single_line.jsonl > /dev/null

```

Après redémarrage ou relecture de Logstash, les nouvelles lignes sont ingérées et traitées par le filtre `aggregate`.

9.4.2 Observation dans Logstash

La commande suivante permet de suivre en temps réel les événements `auth_attempt` et de repérer les marquages `bruteforce` :

```

1 sudo journalctl -u logstash -f | grep -E 'auth_attempt|bruteforce'

```

Listing 9.4 Suivi des événements `auth_attempt` et brute-force

Un extrait représentatif montre, par exemple :

```

1 "message" => "{\"event\":\"auth_attempt\",\"src_ip\":\"127.0.0.1\", ... }",
2 "src_ip" => "127.0.0.1",
3 "event" => [ ["original", "{...}"], "auth_attempt" ],
4 "doc" => {
5     "event" => "auth_attempt",
6     "src_ip" => "127.0.0.1",

```

```

7   "username" => "test",
8   "password" => "kali",
9   "success" => true,
10  ...
11 },
12 "bruteforce" => true,
13 "bruteforce_first_ts" => 2025-11-23T13:13:06.000Z,
14 "bruteforce_count" => 8,
15 "bruteforce_ip" => "127.0.0.1",
16 "@timestamp" => 2025-11-23T13:13:15.000Z

```

Listing 9.5 Exemple d'événements brute-force vus par Logstash

On constate que :

- les premiers `auth_attempt` ne portent pas le flag `bruteforce` (compteur < 5) ;
- à partir de la 5^e tentative, le champ `bruteforce:true` est ajouté, ainsi qu'un compteur `bruteforce_count` croissant (5, 6, 7, 8, ...);
- le même comportement est observé pour plusieurs séries de tentatives sur l'adresse IP 127.0.0.1.

Une capture d'écran de ce suivi en temps réel a été ajoutée au rapport (figure 9.1) :

```

(kali@kali)-[~]
└─$ sudo journalctl -u logstash -f | grep -E 'auth_attempt|bruteforce'

[sudo] password for kali:
Nov 23 10:12:06 kali logstash[213565]:      "message" => "{\"event\":\"auth_attempt\",\"src_ip\":\"127.0.0.1\", \"src_port\":\"43786\", \"username\":\"test\", \"password\":\"kali\", \"success\":true, \"timestamp\":\"2025-11-23T15:11:52Z\"}"
Nov 23 10:12:06 kali logstash[213565]:      [1] "{\"event\":\"auth_attempt\",\"src_ip\":\"127.0.0.1\", \"src_port\":\"43786\", \"username\":\"test\", \"password\":\"kali\", \"success\":true, \"timestamp\":\"2025-11-23T15:11:52Z\"}"
Nov 23 10:12:06 kali logstash[213565]:      [1] "auth_attempt"
Nov 23 10:12:06 kali logstash[213565]:      "event" => "auth_attempt"
Nov 23 10:12:06 kali logstash[213565]:      "message" => "{\"event\":\"auth_attempt\",\"src_ip\":\"127.0.0.1\", \"src_port\":\"43792\", \"username\":\"test\", \"password\":\"kali\", \"success\":true, \"timestamp\":\"2025-11-23T15:11:52Z\"}"
Nov 23 10:12:06 kali logstash[213565]:      [1] "{\"event\":\"auth_attempt\",\"src_ip\":\"127.0.0.1\", \"src_port\":\"43792\", \"username\":\"test\", \"password\":\"kali\", \"success\":true, \"timestamp\":\"2025-11-23T15:11:52Z\"}"
Nov 23 10:12:06 kali logstash[213565]:      [1] "auth_attempt"
Nov 23 10:12:06 kali logstash[213565]:      "event" => "auth_attempt"
Nov 23 10:12:06 kali logstash[213565]:      "message" => "{\"event\":\"auth_attempt\",\"src_ip\":\"127.0.0.1\", \"src_port\":\"43822\", \"username\":\"test\", \"password\":\"kali\", \"success\":true, \"timestamp\":\"2025-11-23T15:11:52Z\"}"
Nov 23 10:12:06 kali logstash[213565]:      [1] "{\"event\":\"auth_attempt\",\"src_ip\":\"127.0.0.1\", \"src_port\":\"43822\", \"username\":\"test\", \"password\":\"kali\", \"success\":true, \"timestamp\":\"2025-11-23T15:11:52Z\"}"
Nov 23 10:12:06 kali logstash[213565]:      [1] "auth_attempt"
Nov 23 10:12:06 kali logstash[213565]:      "event" => "auth_attempt"
Nov 23 10:12:06 kali logstash[213565]:      "message" => "{\"event\":\"auth_attempt\",\"src_ip\":\"127.0.0.1\", \"src_port\":\"43824\", \"username\":\"test\", \"password\":\"kali\", \"success\":true, \"timestamp\":\"2025-11-23T15:11:53Z\"}"
Nov 23 10:12:06 kali logstash[213565]:      [1] "auth_attempt"
Nov 23 10:12:06 kali logstash[213565]:      "event" => "auth_attempt"
Nov 23 10:12:06 kali logstash[213565]:      "message" => "{\"event\":\"auth_attempt\",\"src_ip\":\"127.0.0.1\", \"src_port\":\"43844\", \"username\":\"test\", \"password\":\"kali\", \"success\":true, \"timestamp\":\"2025-11-23T15:11:54Z\"}"
Nov 23 10:12:06 kali logstash[213565]:      "bruteforce" => true,
Nov 23 10:12:06 kali logstash[213565]:      "bruteforce_first_ts" => 2025-11-23T15:11:52.000Z,
Nov 23 10:12:06 kali logstash[213565]:      [1] "{\"event\":\"auth_attempt\",\"src_ip\":\"127.0.0.1\", \"src_port\":\"43844\", \"username\":\"test\", \"password\":\"kali\", \"success\":true, \"timestamp\":\"2025-11-23T15:11:54Z\"}"
Nov 23 10:12:06 kali logstash[213565]:      [1] "auth_attempt"
Nov 23 10:12:06 kali logstash[213565]:      "event" => "auth_attempt"
Nov 23 10:12:06 kali logstash[213565]:      "bruteforce_count" => 5,
Nov 23 10:12:06 kali logstash[213565]:      "bruteforce_ip" => "127.0.0.1"

```

Figure 9.1 Affichage des événements `auth_attempt` marqués `bruteforce:true` dans les logs Logstash (`journalctl -u logstash -f`).

9.4.3 Vérification dans Elasticsearch et Kibana

Côté Elasticsearch, les événements marqués peuvent être retrouvés via une requête simple :

```
1 curl -s "http://127.0.0.1:9200/honeypot-ssh-clean-*/_search?q=bruteforce:true&sort=@timestamp:desc&size=10" \  
2 | jq '.hits.hits[]._source | {ts: .["@timestamp"], ip: .bruteforce_ip, count: .bruteforce_count}'
```

Listing 9.6 Recherche des événements brute-force dans Elasticsearch

Le résultat typique contient des documents du jour portant le flag `bruteforce:true` :

```
1 {  
2   "ts": "2025-11-23T13:13:15.000Z",  
3   "ip": "127.0.0.1",  
4   "count": 5  
5 }  
6 {  
7   "ts": "2025-11-23T13:13:15.000Z",  
8   "ip": "127.0.0.1",  
9   "count": 6  
10 }  
11 ...
```

Listing 9.7 Exemple d'événements brute-force dans Elasticsearch

Dans Kibana, un filtre KQL `bruteforce : true` appliqué sur la data view `honeypot-ssh-clean` permet d'obtenir instantanément la liste des événements associés et de les intégrer à un dashboard dédié (visualisation du nombre d'alertes par IP, par période, etc.).

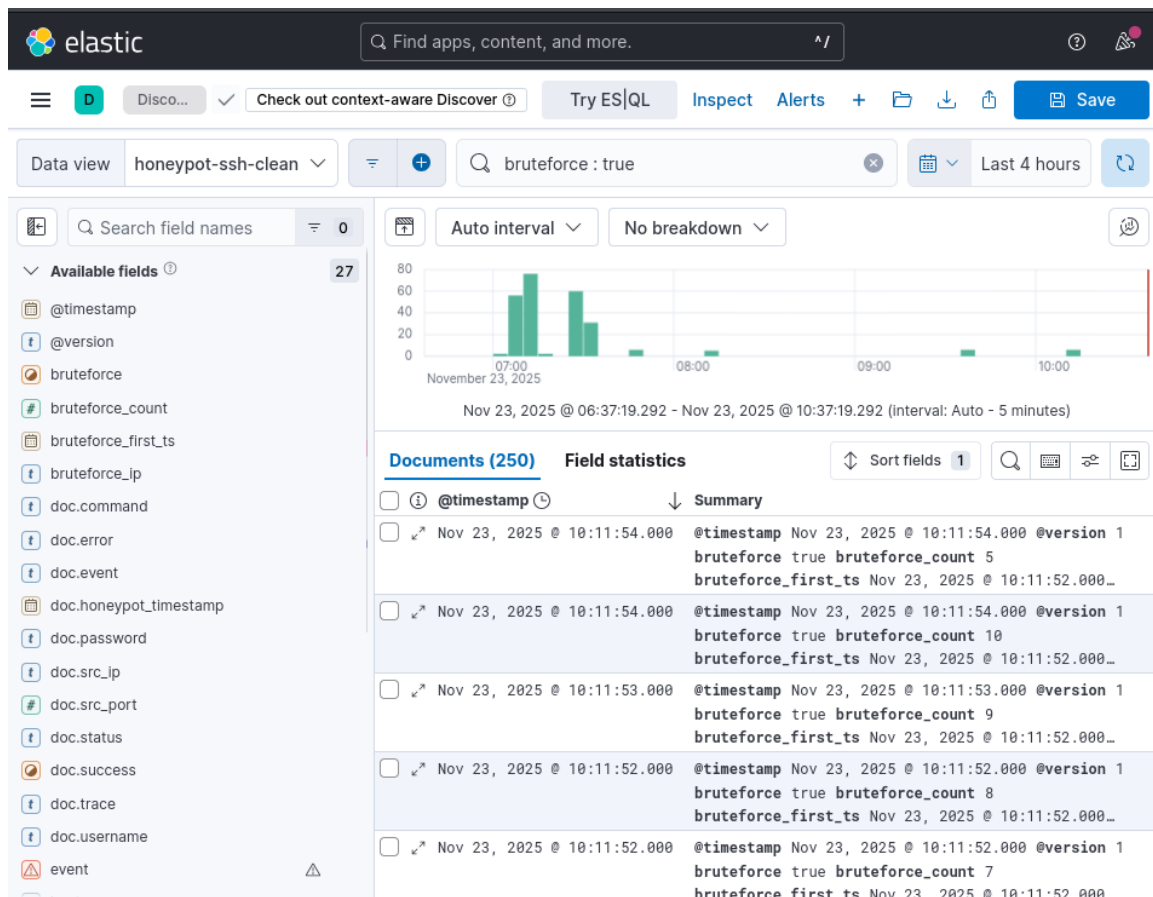


Figure 9.2 Filtrage des événements `bruteforce:true` dans Kibana (Discover, data view `honeypot-ssh-clean`).

9.5 Tests avec Hydra et vérification du confinement

9.5.1 Tentatives avec Hydra

L'outil `hydra` a été utilisé pour tenter de générer un trafic de brute-force plus réaliste :

```
1 hydra -l test -p badpassword -t 1 -s 2222 -V 127.0.0.1 ssh
```

Listing 9.8 Exemple de commande Hydra utilisée

Toutefois, en pratique, certaines combinaisons de paramètres entraînent des erreurs de protocole côté Paramiko (par exemple `MessageOrderError: Expecting packet from (30,), got 34`). Ces erreurs sont correctement journalisées dans les événements `server_error` du honeypot, mais rendent le scénario moins stable que la boucle `sshpas`. Pour des tests reproductibles et faciles à interpréter, `sshpas` a donc été privilégié, et Hydra n'a servi que de validation ponctuelle.

9.5.2 Vérification du profil AppArmor en situation d'attaque

Pendant l'exécution des tests de brute-force, le honeypot est systématiquement lancé via :

```
1 cd /opt/ssh_honeypot
2 sudo aa-exec -p honeypot-ssh -- ./venv/bin/python ssh_honeypot.py --port 2222
```

Listing 9.9 Lancement du honeypot sous profil AppArmor

La commande `ps auxZ` confirme le profil AppArmor appliqué :

```
1 ps auxZ | grep ssh_honeypot.py
2
3 honeypot-ssh (complain) root ... ./venv/bin/python ssh_honeypot.py --port 2222
```

Listing 9.10 Vérification du confinement AppArmor

Des tentatives d'accès à des fichiers interdits (par exemple `/etc/shadow`, `/proc/self/environ`) sont ensuite réalisées depuis le faux shell du honeypot ou via des commandes simulées. Les violations correspondantes apparaissent dans les logs du noyau :

```
1 sudo journalctl -k | grep -i apparmor | tail -n 10
```

Listing 9.11 Exemple de violations AppArmor pendant les tests

En résumé, le bonus de détection de brute-force est pleinement fonctionnel : les vagues de tentatives d'authentification sont détectées et marquées dans les événements, exploitables ensuite dans Elasticsearch/Kibana et, si souhaité, envoyées à un webhook externe. Le tout est testé en conditions réalistes avec un honeypot confiné par AppArmor et protégé par un filtre Seccomp.

10. Conclusion

Ce projet a permis de mettre en œuvre une solution complète de détection et de supervision d'un service exposé :

- Un honeypot SSH en Python, basé sur `paramiko`, enregistre de manière détaillée les tentatives d'authentification et les commandes exécutées.
- Le processus du honeypot est confiné par AppArmor (profil dédié) et Seccomp (filtrage d'appels système), ce qui réduit fortement l'impact potentiel d'une compromission.
- Les événements sont collectés, transformés et indexés par Logstash et Elasticsearch dans des indices quotidiens `honeypot-ssh-clean-*`.
- Kibana permet de visualiser et d'explorer les données via un data view et un dashboard dédiés.

L'approche multi-couches combine des mécanismes de sécurité du noyau (AppArmor, Seccomp) et une stack de supervision puissante (ELK). Cette architecture peut servir de base à des travaux ultérieurs, notamment :

- enrichissement de la détection de brute-force (seuils dynamiques, corrélation multi-services) et création de dashboards d'alertes ;
- envoi d'alertes via webhook, e-mail ou intégration avec un SIEM ;
- enrichissement des événements avec des informations géographiques (GeoIP) pour des visualisations cartographiques dans Kibana.

Dans le cadre de ce projet, une première version de détection de brute-force a d'ailleurs été réalisée : les tentatives d'authentification sont corrélées dans Logstash à l'aide du filtre `aggregate`, qui marque automatiquement les événements dépassant le seuil avec un drapeau `bruteforce:true`. Ces événements sont ensuite visibles dans Elasticsearch et filtrables dans Kibana, ce qui valide la faisabilité de mécanismes d'alerte plus avancés (webhook, notifications externes, intégration SIEM).

De la même manière, les tests de charge réalisés à l'aide d'outils comme `sshpas` et `hydra`, combinés à la vérification systématique du profil AppArmor appliqué au processus du honeypot, montrent que la plate-forme reste confinée même en situation d'attaque et que les mécanismes de supervision restent opérationnels.

Au final, ce travail illustre comment il est possible, avec des composants libres relativement simples, de construire une chaîne complète allant de la capture d'activité malveillante (honeypot) jusqu'à son analyse structurée (ELK), en passant par un durcissement concret du service exposé (AppArmor, Seccomp) et une première couche de détection de comportements anormaux (brute-force).